Laboratorio 1

Introducción

Un escáner (o analizador léxico) constituye la primera fase de un compilador. Su función principal consiste en leer el código fuente como una secuencia de caracteres y agruparlos en unidades léxicas denominadas *tokens*. Cada token representa un símbolo significativo dentro del lenguaje, tales como operadores aritméticos, números enteros, paréntesis, identificadores o palabras reservadas. Para realizar esta tarea, el escáner suele implementarse mediante un **autómata finito** que recorre la entrada carácter por carácter y reconoce patrones definidos por el lenguaje. Como resultado, transforma el texto original en una secuencia estructurada de tokens que posteriormente será utilizada por el analizador sintáctico y el analizador semántico en las siguientes fases del proceso de compilación. Por ejemplo, la expresión aritmética:

$$3 + 5 * 2$$

es convertida en la siguiente secuencia de tokens:

- TOKEN(NUM, "3")
- TOKEN(PLUS, -")
- TOKEN(NUM, "5")
- TOKEN(MUL, "*")
- TOKEN(NUM, "2")
- TOKEN(END)

De esta manera, el escáner actúa como un puente entre el texto fuente y las etapas posteriores del compilador, permitiendo que la entrada sea interpretada como una secuencia ordenada de elementos con significado dentro del lenguaje de programación o expresión matemática que se desea procesar.

Objetivo del laboratorio

- Comprender el rol del escáner dentro de un compilador.
- Analizar cómo un autómata finito implementado en C++ reconoce tokens.
- Ejecutar el escáner sobre distintas entradas y observar la secuencia de tokens producida.
- Extender el escáner.

Archivos principales

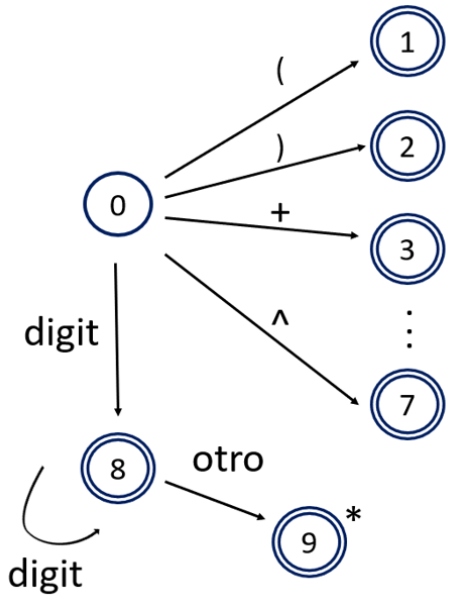
`token.h`

- Este archivo define la clase `Token`, la cual encapsula la representación de cada unidad léxica reconocida por el escáner.
- Un token combina dos aspectos esenciales: el tipo que lo clasifica dentro de la gramática del lenguaje y el lexema que le da valor.
- Tipos de tokens: se encuentran organizados en una enumeración (`enum Type`), que incluye las siguientes categorías básicas:
  - Operadores aritméticos: `PLUS (+)`, `MINUS (-)`, `MUL (*)`, `DIV (/)`, `POW (^)`.
  - Delimitadores: `LPAREN (`, `RPAREN )`.
  - Otros: `NUM` (número entero), `ERR` (símbolo no válido o error léxico), `END` (fin de la entrada).
- Atributos principales:
  - `Type type`: indica la categoría a la que pertenece el token.
  - `string text`: almacena el lexema o el fragmento exacto de la cadena de entrada que fue identificado.

`scanner.h`

- Este archivo define la clase `Scanner`, responsable de recorrer la cadena de entrada y segmentarla en unidades léxicas, generando instancias de la clase `Token`.
- El escáner implementa un autómata finito que avanza carácter por carácter, identificando patrones válidos y reportando errores cuando corresponda.
- Atributos principales:
  - `string input`: almacena el texto completo que se va a analizar.
  - `int first, current`: índices que marcan el inicio y la posición actual en la cadena, necesarios para delimitar lexemas.
  - `int state`: representa el estado actual del autómata durante el análisis.
- Métodos principales:
  - `nextToken()`: existen dos versiones de este método. La primera está basada en un autómata finito, que utiliza los métodos `nextChar()` y `rollBack()` para controlar el avance y retroceso en la lectura de caracteres. La segunda versión está elaborada de manera más intuitiva, sin depender directamente del autómata.

- `nextChar()`: obtiene el siguiente carácter de la cadena de entrada (utilizado en la versión basada en autómata).
- `rollBack()`: retrocede una posición en caso de lectura innecesaria o error (utilizado en la versión basada en autómata).
- `ejecutar_scanner()`: permite ejecutar el proceso completo del escáner sobre un archivo de texto y mostrar la secuencia de tokens generados.



Ejecución del escáner

Para ejecutar el escáner se incluye el archivo `run_all_inputs.py`, un script en Python que primero compila los archivos `main.cpp`, `scanner.cpp` y `token.cpp` con `g++`, y luego ejecuta el programa sobre diferentes archivos de entrada ubicados en la carpeta `inputs`. De esta forma, el proceso de compilación y prueba se automatiza, permitiendo verificar rápidamente el funcionamiento del escáner con varios casos de prueba. Cabe señalar que, dependiendo del sistema operativo, el ejecutable generado por `g++` puede ser `a.exe` (Windows) o `a.out` (Linux/MacOS), por lo que el script debe adaptarse en consecuencia.

Ejercicios

Todas las modificaciones indicadas deben implementarse de manera que sean compatibles con ambas versiones del método `nextToken()` (la basada en autómata y la elaborada de manera intuitiva).

1. Modificar el operador de potencia reemplazando `^` por `**`.
2. Entrada multilínea: adaptar el analizador léxico para procesar un archivo que contenga múltiples sentencias separadas por punto y coma.
3. Nuevos tokens: extender el autómata y el escáner para reconocer números de punto flotante e identificadores. Se añadirán dos nuevos tokens: `FLOAT` e `ID`. Las expresiones regulares correspondientes son:
   - `float ::= digit digit* '.' digit*`
   - `id ::= alpha (alpha | digit)*`
   - `digit ::= 0 | 1 | ... | 9`
   - `alpha ::= a ... z | A ... Z`
4. Funciones y constantes: las expresiones aritméticas pueden incluir funciones predefinidas (`SIN`, `COS`, `LOG`, etc.) y constantes (`PI`, `E`). Dado que tanto las funciones como las constantes siguen las mismas reglas de formación de identificadores, se debe explicar cómo generar estos nuevos tokens sin modificar el autómata, aplicable a ambas versiones de `nextToken()`.
5. Modificar los tokens para que además de su tipo y lexema, guarden información de posición en el código fuente, incluyendo número de línea y columna.
6. Comentarios: extender el escáner para reconocer comentarios en el código fuente. Los comentarios pueden ser de línea (`//`) o multilínea (`/* ... */`) y deben ser ignorados durante el análisis, sin generar tokens.

Conclusiones

El escáner implementado permite:

- Traducir una cadena de caracteres en una secuencia de tokens.
- Manejar operadores, números y delimitadores básicos.
- Servir como **primer bloque en la construcción de un compilador o intérprete**.
- Ser ampliado para reconocer identificadores, palabras reservadas y números flotantes.